FPT University

COS201

PARITY ERROR DETECTOR

Design and verification of an 8-bit even parity generator, error injector and checker in Verilog

Student name: Bùi Quang Vinh Vương

Student ID: SE204292

Student name: Đỗ Phước Hùng

Student ID: SE204688

Student name: Nguyễn Nam Khánh

Student ID: SE205204

Student name: Vũ Minh Quân

Student ID: SE204813

Student name: Trương Ngọc Anh Khoa

Student ID: SE205096

Student name : Nguyễn Lâm Văn
Student ID : SE204977

1. Introduction

When data is sent from one place to another, some bits can change because of noise or a fault in the line. The receiver needs a way to know that the data it got is not the same as the data that was sent. A very simple way to do this is to add a parity bit to the data.

In this project we design a parity error detector for 8-bit data in Verilog. The transmitter makes an even parity bit from the data. The transmission channel is modeled by an error injector, which can flip one bit of the data when we want. The receiver checks the received data together with the parity bit and gives an error flag.

The main goals of the project are:

- Design the parity generator, the error injector and the parity checker as separate modules.
- Connect the three modules in one top module.
- Test every module and the top module with its own testbench in ModelSim.
- Write a Python golden model and compare its results with the Verilog design.

2. Background

2.1 Even parity

A parity bit is one extra bit that is added to the data so that the number of 1s follows a rule. With even parity, the total number of 1s in the data and the parity bit together must be even. So if the data has an odd number of 1s the parity bit is 1, and if it has an even number of 1s the parity bit is 0.

For example, the data 10110010 has four 1s, so its parity bit is 0. The data 01110000 has three 1s, so its parity bit is 1.

2.2 How an error is detected

The receiver counts the 1s again, this time including the parity bit. If the count is still even, no error is found. If the count is odd, at least one bit was changed on the way. Counting the 1s in this way is the same as taking the XOR of all the bits, because the XOR of a group of bits is 1 when the number of 1s is odd. This is why the generator and the checker are both just XOR circuits.

2.3 Limits of parity

Parity can only detect an odd number of wrong bits (1, 3, 5, ...). If two bits are flipped at the same time the number of 1s stays even, so the error is not seen. Parity also does not say which bit is wrong, so it cannot correct the data. In this project we only flip a single bit, so the checker can always detect the error [4], [5].

3. System Design

3.1 System flow

Figure 1 shows how the data moves through the system. The input data goes to the parity generator (transmitter) and then through the transmission channel. At the receiver, the parity checker looks at the received data and the parity bit and gives the result: error_flag = 0 means the data is OK, and error_flag = 1 means an error was found.

In our design the transmission channel is the error injector. It only changes the data bits. The parity bit goes directly from the generator to the checker, so only the data can be damaged.

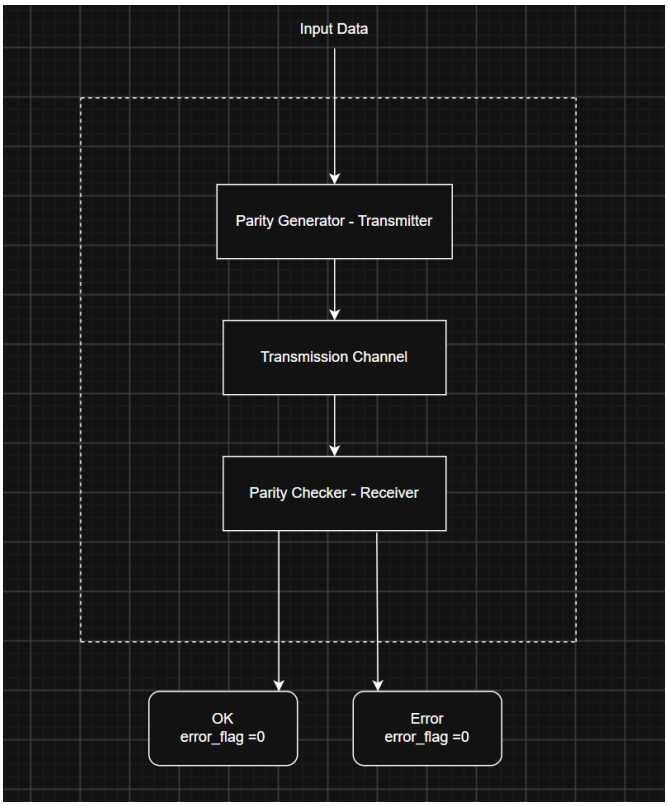


Figure 1. System flow of the parity error detector

3.2 High-level design

Figure 2 is the high-level block diagram. The top module has two inputs, data_in[7:0] and error_enable. Inside, data_in goes to both the parity generator and the error injector. The generator makes parity_bit, and the injector makes data_rx, which is the data after the channel. The parity checker takes data_rx and the parity bit (called parity_rx at the checker input) and produces error_flag.

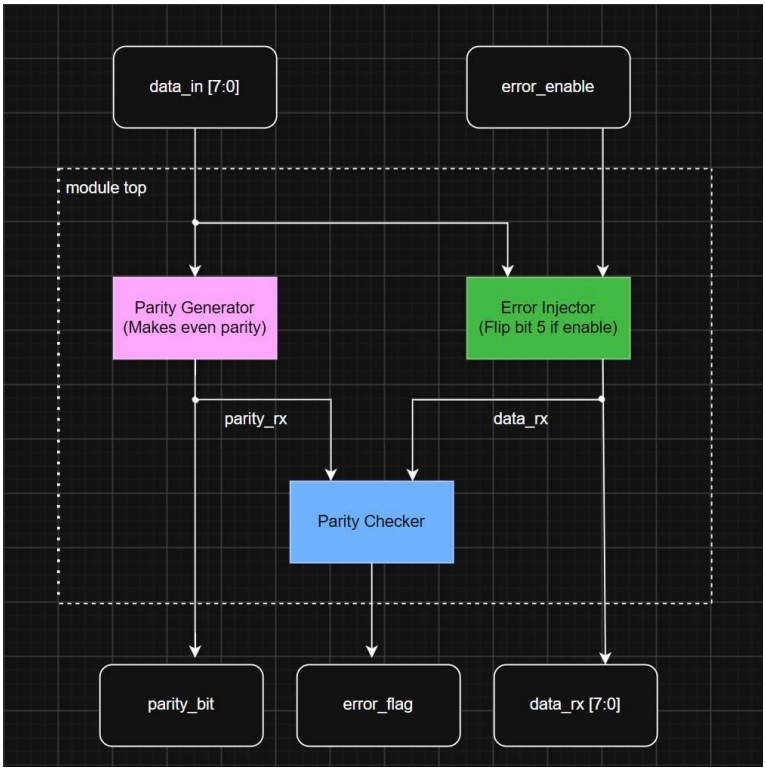


Figure 2. High-level block diagram

3.3 Low-level design

Figure 3 shows the same design at bit level. In the parity generator, the 8 bits of data_in go into one XOR block. In the error injector, bits 0 to 4, 6 and 7 pass straight through. Only bit 5 goes through an XOR gate with error_enable, so this bit is flipped when error_enable is 1 and left alone when it is 0. In the parity checker, the 8 bits of data_rx and the parity bit are XORed together to get error_flag.

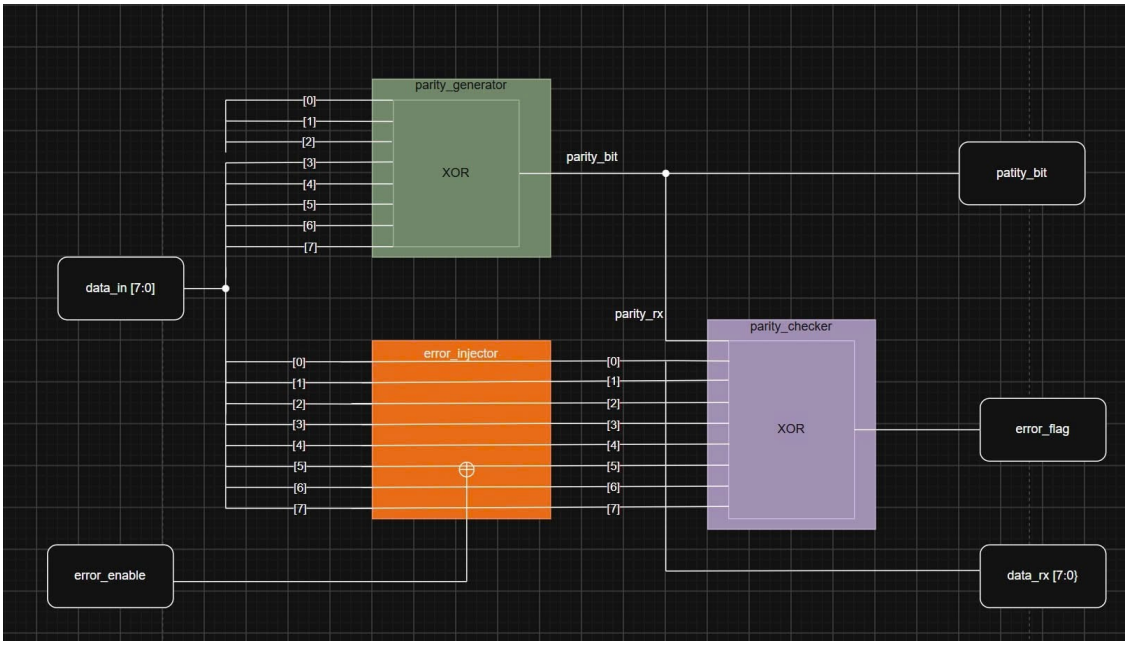


Figure 3. Low-level diagram

3.4 Top module interface

Table 1 lists the ports of the top module. The signal ok_flag is just the opposite of error_flag. It is added to make the result easier to read in the waveform.

Table 1. Ports of the top module

Signal	Direction	Width	Description
data_in	Input	8	Data to be sent
error_enable	Input	1	1 = flip bit 5 of the data, 0 = no error
parity_bit	Output	1	Even parity bit made from data_in
data_rx	Output	8	Data after the error injector
ok_flag	Output	1	1 = no error found
error_flag	Output	1	1 = error found

4. RTL Implementation

The design has four Verilog files: three small modules and the top module that connects them.

4.1 Parity generator

The parity generator XORs the 8 bits of data_in. The result is 1 when the data has an odd number of 1s, which is exactly the even parity bit.

```
module parity_generator (
    input wire [7:0] data_in,
    output reg parity_bit
);
always @(*) begin
    parity_bit = data_in[0] ^ data_in[1] ^ data_in[2] ^ data_in[3] ^
                 data_in[4] ^ data_in[5] ^ data_in[6] ^ data_in[7];
end
endmodule
```

4.2 Error injector

The error injector copies data_in to data_out. If error_enable is 1, bit 5 is inverted. This models a single bit error in the channel.

```
module error_injector (
    input wire [7:0] data_in,
    input wire error_enable,
    output reg [7:0] data_out
);
always @(*) begin
    if (error_enable == 1'b0) begin
        data_out = data_in;
    end
    else begin
        data_out = data_in;
        data_out[5] = ~data_in[5];
    end
end
endmodule
```

4.3 Parity checker

The parity checker XORs the 8 received bits and the received parity bit. If the result is 0 the number of 1s is even and the data is accepted. If the result is 1 the number of 1s is odd, so an error is reported.

```
module parity_checker (
    input wire [7:0] data_rx,
    input wire parity_rx,
    output reg error_flag
);
always @(*) begin
    error_flag = data_rx[0] ^ data_rx[1] ^ data_rx[2] ^ data_rx[3] ^
                 data_rx[4] ^ data_rx[5] ^ data_rx[6] ^ data_rx[7] ^
                 parity_rx;
end
endmodule
```

4.4 Top module

The top module connects the three modules as in the system flow. The parity bit from the generator goes to the checker, the output of the error injector goes to the checker as data_rx, and ok_flag is made by inverting error_flag.

```
module top (
    input wire [7:0] data_in,
    input wire error_enable,
    output wire parity_bit,
    output wire [7:0] data_rx,
    output wire ok_flag,
    output wire error_flag
);

    parity_generator U1 (
        .data_in(data_in),
        .parity_bit(parity_bit)
    );

    error_injector U2 (
        .data_in(data_in),
        .error_enable(error_enable),
        .data_out(data_rx)
    );

    parity_checker U3 (
        .data_rx(data_rx),
        .parity_rx(parity_bit),
        .error_flag(error_flag)
    );

    assign ok_flag = ~error_flag;

endmodule
```

5. Verification

5.1 Environment and method

The Verilog files were simulated in ModelSim Intel FPGA Starter Edition 2020.1. Each module has its own testbench (tb_parity_generator, tb_error_injector, tb_parity_checker and tb_top). In each testbench a task called check puts a value on the inputs, waits 10 ns, compares the output with the expected value and prints PASS or FAIL. The expected value is calculated inside the testbench, so the result does not depend on the design being tested.

The golden model is a Python program. It does the same steps in a different way and gives a second result to compare with.

5.2 Test data

All testbenches use the same five test cases. The values 178, 255 and 0 are used because their parity bit is easy to check by hand: 178 (10110010) has four 1s, 255 (11111111) has eight 1s

and 0 has none, so the parity bit is 0 in all three cases. Each of the first two values is tested once without error and once with error. Table 2 gives the expected result.

Table 2. Test cases and expected results

No.	data_in	error_enable	parity_bit	data_rx	error_flag
1	10110010	0	0	10110010	0
2	10110010	1	0	10010010	1
3	11111111	0	0	11111111	0
4	11111111	1	0	11011111	1
5	00000000	0	0	00000000	0

5.3 Parity generator

The testbench of the parity generator uses the five values above and three more values that have an odd number of 1s: 00000001, 10000000 and 01110000. For these the parity bit must be 1. The expected value in the testbench is calculated with the reduction XOR operator (^d). The result is shown in Table 3 and Figure 4.

Table 3. Parity generator test cases

No.	data_in	Expected parity_bit
1	10110010	0
2	10110010	0
3	11111111	0
4	11111111	0
5	00000000	0
6	00000001	1
7	10000000	1
8	01110000	1

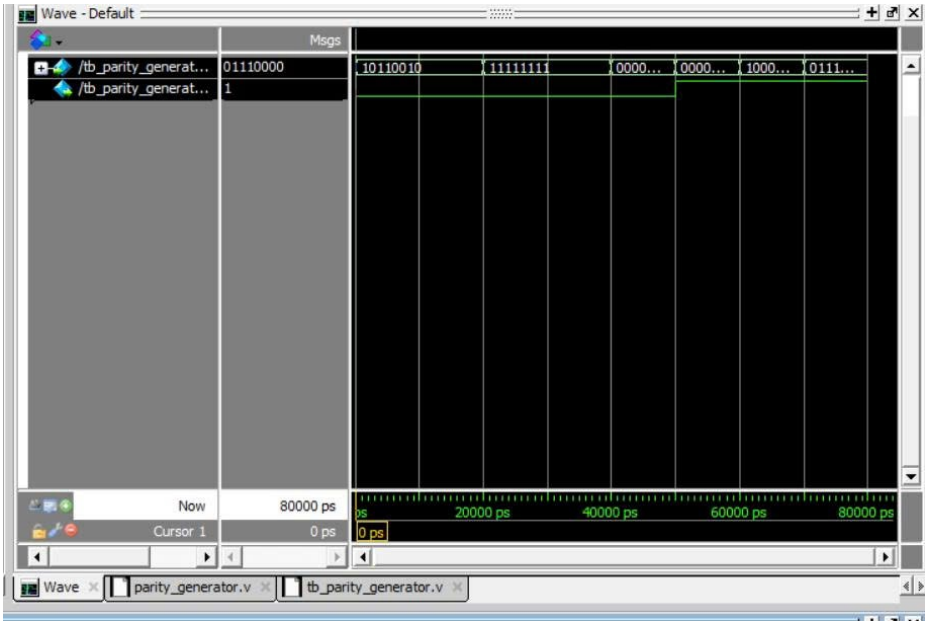


Figure 4. Waveform of the parity generator

5.4 Error injector

For the error injector the testbench builds the expected output by copying the input and flipping bit 5 when error_enable is 1. In the waveform (Figure 5) data_out is the same as data_in when error_enable is 0, and bit 5 is flipped when it is 1. For example 10110010 becomes 10010010 and 11111111 becomes 11011111.

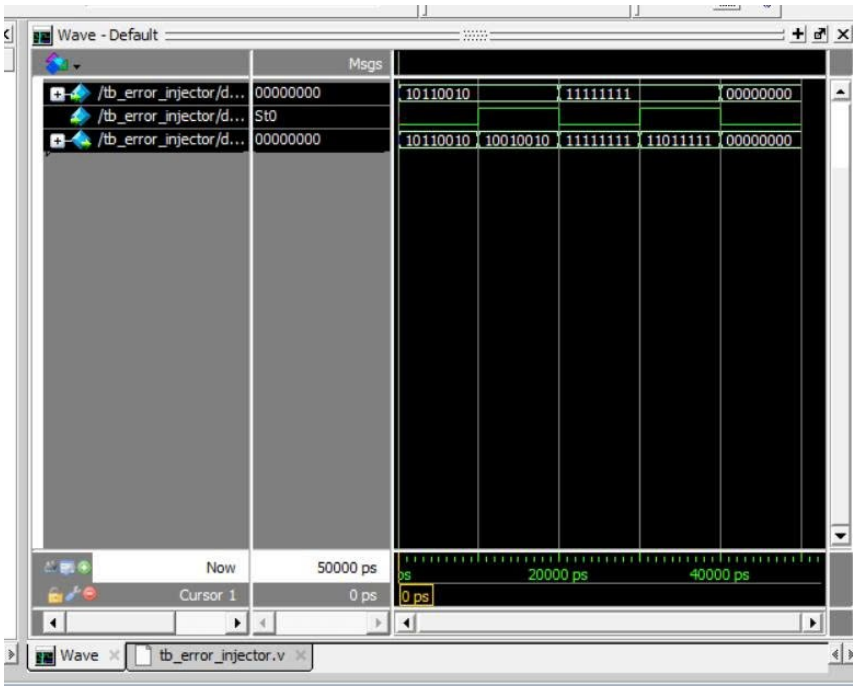


Figure 5. Waveform of the error injector

5.5 Parity checker

The parity checker is tested with data_rx and parity_rx given directly. Cases 1, 3 and 5 are data without error, so error_flag must be 0. Cases 2 and 4 have one bit changed, so error_flag must be 1. The result is shown in Table 4 and Figure 6.

Table 4. Parity checker test cases

No.	data_rx	parity_rx	Expected error_flag
1	10110010	0	0
2	10010010	0	1
3	11111111	0	0
4	11011111	0	1
5	00000000	0	0

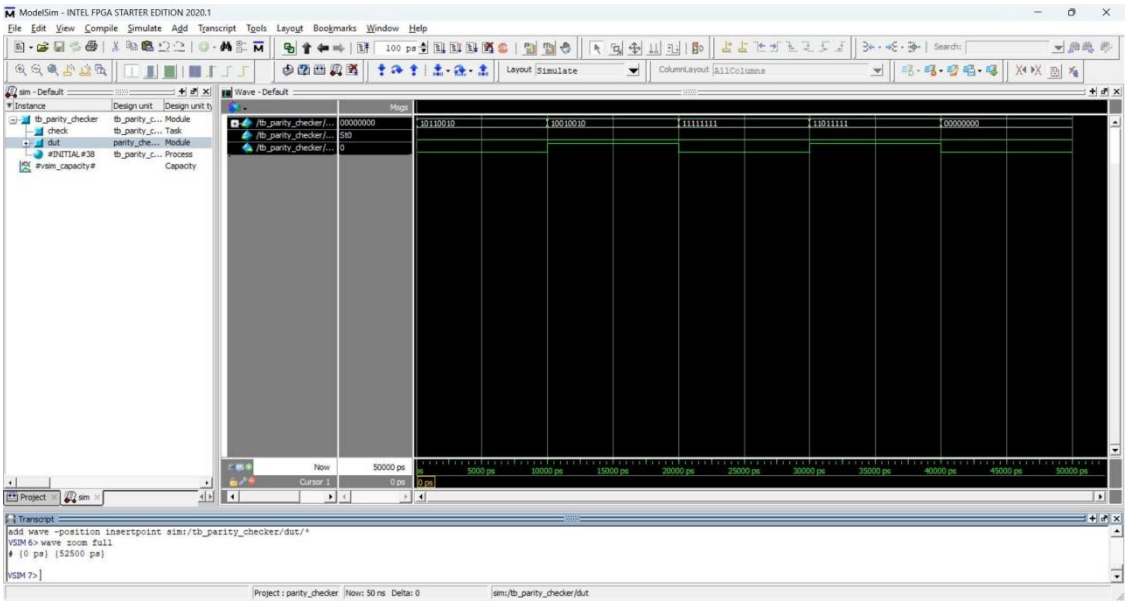


Figure 6. Waveform of the parity checker

5.6 Top module

The top testbench runs the five cases of Table 2 on the full system. It checks that error_flag is equal to error_enable and that ok_flag is the opposite of error_flag. In Figure 7, parity_bit stays at 0 for all five inputs, data_rx changes only in cases 2 and 4, and error_flag is 1 only in those two cases. ok_flag is always the opposite of error_flag.

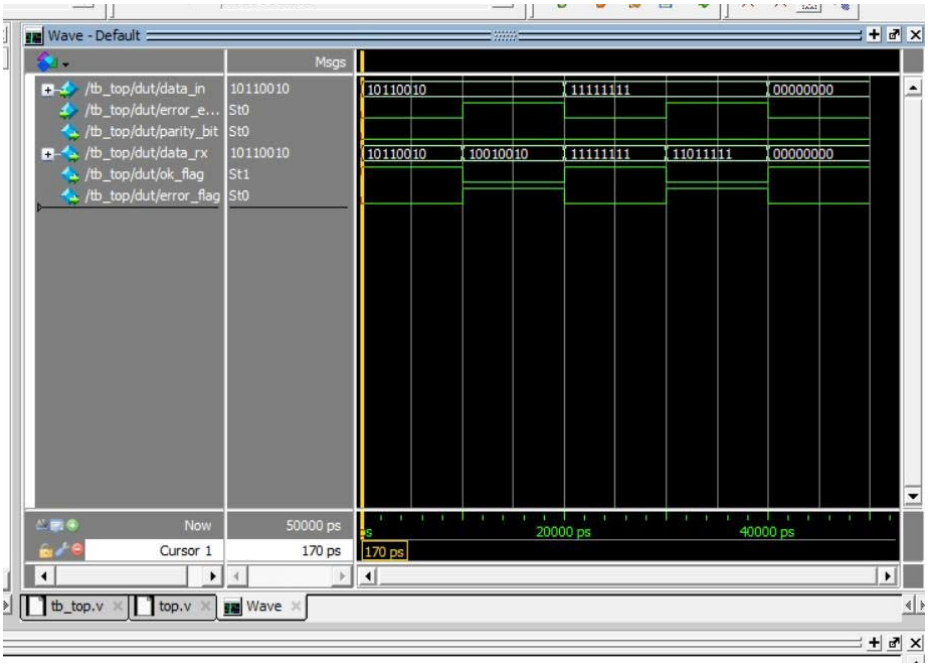


Figure 7. Waveform of the top module

5.7 Golden model

The golden model is written in Python without using the XOR operator. It follows these steps for each test case:

- Change the number into 8 bits.
- Count the 1s and set parity_bit to 1 if the count is odd, otherwise 0.
- Flip one bit of the data if error_enable is 1.
- Count the 1s in the received data plus the parity bit. If the count is odd, error_flag is 1, otherwise 0.
- Compare error_flag with error_enable and print PASS or FAIL.

The output of the program is shown in Figure 8 and summarized in Table 5. All five cases are PASS.

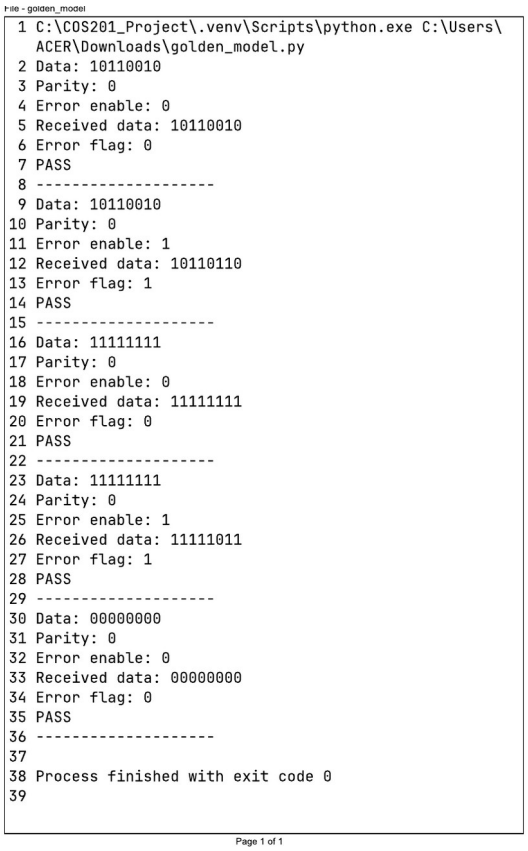


Figure 8. Output of the golden model

Table 5. Golden model results

No.	Data	error_enable	Parity	Received data	error_flag	Result
1	10110010	0	0	10110010	0	PASS
2	10110010	1	0	10110110	1	PASS
3	11111111	0	0	11111111	0	PASS
4	11111111	1	0	11111011	1	PASS
5	00000000	0	0	00000000	0	PASS

The parity bit and error_flag of the golden model are the same as in the Verilog simulation for every case. One small difference is the position of the flipped bit. The Python program stores the bits from left to right, so index 5 is the sixth bit from the left, while data_out[5] in Verilog is the sixth bit from the right. Because of this, the received data in Table 5 is not the same as in Table 2 for the error cases. In both models exactly one bit is flipped, so error_flag is the same and the comparison still passes.

6. Conclusion

In this project we designed a parity error detector for 8-bit data. The parity generator makes an even parity bit, the error injector flips bit 5 when we want, and the parity checker finds the error.

Each module and the top module were tested in ModelSim, and the results match the expected values and the Python golden model.

The design is simple, but it has the usual limits of parity: it can only detect an odd number of wrong bits and it cannot correct them. The tests also cover only a few data values and a single bit error.

Some things that could be done next:

- Test all 256 values of data_in, and make the injector flip other bits.
- Test a two-bit error to show that parity cannot detect it.
- Let the error also change the parity bit.

7. References

[1] 8-bit Parity Generator and Checker in Verilog, Engineers Garage. https://www.engineersgarage.com/verilog-tutorial-12-how-to-design-8-bit-parity-generator-and-checker-circuits-in-verilog/

[2] Parity Generator and Checker, UPC. https://digsys.upc.edu/csd/products/D1_14/D1_14.html

[3] Design and Simulation of an Even Parity Checker, CMOSedu. https://cmosedu.com/jbaker/courses/ee421L/f17/students/verdum1/proj/proj.htm

[4] Error Detecting and Correcting Codes, University of Michigan. https://www.eecs.umich.edu/courses/eecs373.w05/lecture/errorcode.html

[5] Even Parity - Error Detection, University of New Brunswick. https://www.ece.unb.ca/tervo/ece4253/ecc.shtml

[6] Web Coding Theory Book, Parity Code. https://users.math.msu.edu/users/iwenmark/Teaching/MTH810/web-coding-book2.pdf

[7] Error Detection and Correction, Stanford University. https://web.stanford.edu/class/ee179/lectures/notes18.pdf

[8] Error Detection: Parity Bits and Check Digits, Longwood University. https://cs.longwood.edu/courses/cmsc121/resources/s24/error-detection/

[9] The Mathematics of Coding Theory. https://pws.npru.ac.th/sartthong/data/files/_Mathematics_of_Coding_Theory.pdf

[10] Error Detection and Correction, NPTEL. https://archive.nptel.ac.in/content/storage2/courses/106105080/pdf/M3L2.pdf

[11] Parity Generator and Checker Explained, YouTube. https://www.youtube.com/watch?v=c8qAti1zBVQ

[12] Parity Generator, Paiz Logic. https://docs.paizlogic.com/en/dec-paritygen

[13] Parity Bit Tutorial: Single-Bit Error Detection, DigiSim. https://digisim.io/blog/parity-bit-tutorial-single-bit-error-detection